09. CUDA 가속화 구현 분석

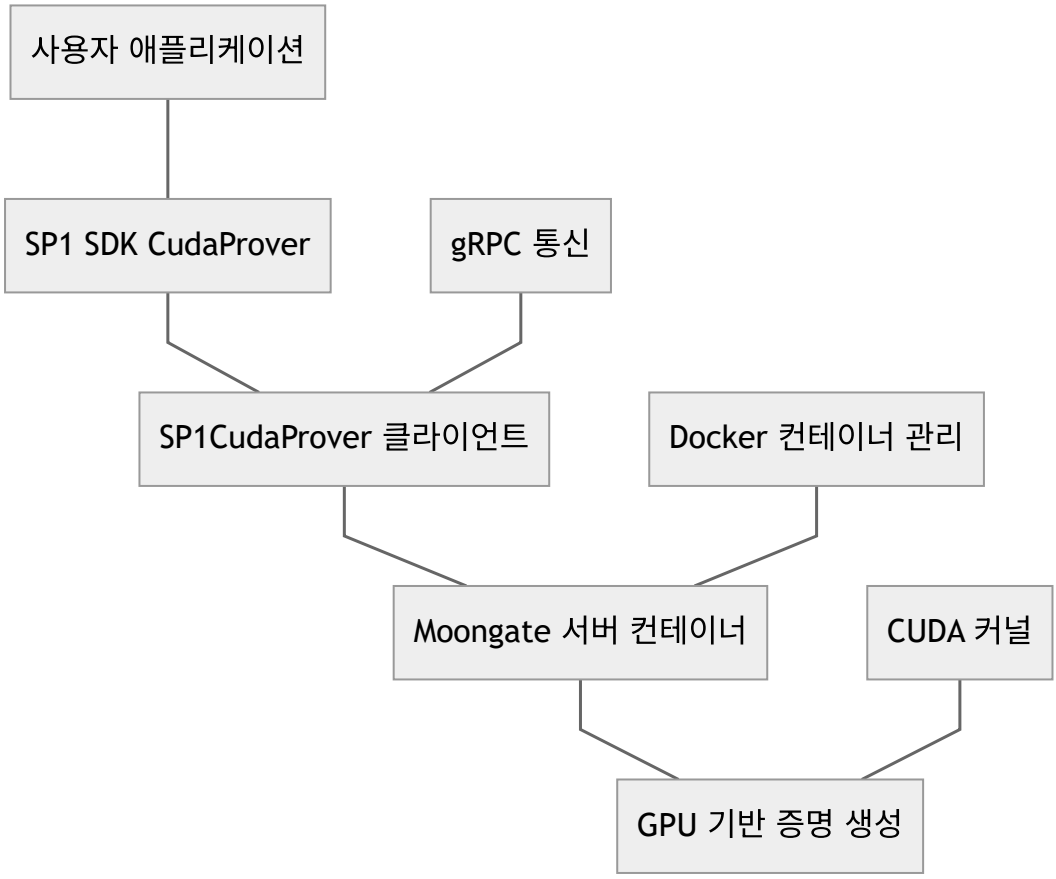
개요

SP1은 GPU 기반 CUDA 가속화를 통해 증명 생성 성능을 대폭 향상시킵니다. 이 문서는 SP1의 CUDA 가속화 아키텍처, 구현 방식, 그리고 성능 향상 효과를 상세히 분석합니다.

CUDA 가속화 아키텍처

전체 구조

SP1의 CUDA 가속화는 다음과 같은 계층적 구조로 구현됩니다:



주요 컴포넌트

1. CudaProver (SDK 레벨)

- **역할**: 사용자 인터페이스 제공 및 CPU/GPU 작업 조정
- **위치**: `crates/sdk/src/cuda/mod.rs`
- **주요 기능**:
- CPU 기반 실행과 GPU 기반 증명 생성 분리
- 다양한 증명 모드 지원 (Core, Compressed, Plonk, Groth16)
- 자동 폴백 메커니즘

2. SP1CudaProver (CUDA 클라이언트)

- **역할**: GPU 서버와의 통신 및 증명 생성 요청 처리
- **위치**: `crates/cuda/src/lib.rs`
- **주요 기능**:
- Moongate 서버 관리
- gRPC 기반 원격 증명 생성
- Docker 컨테이너 자동 관리

3. Moongate 서버

- **역할**: GPU에서 실제 증명 생성 수행
- **배포 방식**: Docker 컨테이너
- **이미지**: `public.ecr.aws/succinct-labs/moongate:v5.0.8`

CUDA 구현 세부사항

서버 초기화 및 관리

```
impl SP1CudaProver {
pub fn new(moongate_server: MoongateServer) -> Result<Self, Box<dyn StdError>>
let prover = match moongate_server {
MoongateServer::External { endpoint } => {
// 외부 서버 연결
let client = Client::new(
Url::parse(&endpoint).expect("failed to parse url"),
reqwest::Client::new(),
reqwest_middlewares,
).expect("failed to create client");
SP1CudaProver { client, managed_container: None }
}
MoongateServer::Local { visible_device_index, port } => {
// 로컬 Docker 컨테이너 시작
Self::start_moongate_server(reqwest_middlewares, visible_device_index, port)?
}
};
// 서버 준비 상태 확인 (최대 300초 대기)
let timeout = Duration::from_secs(300);
// ... 준비 상태 확인 로직
}
```

```
}
```

Docker 컨테이너 관리

SP1은 CUDA 가속화를 위해 Docker 컨테이너를 자동으로 관리합니다:

```
fn start_moongate_server(
reqwest_middlewares: Vec<Box<dyn Middleware>>,
visible_device_index: Option<u64>,
port: Option<u64>,
) -> Result<SP1CudaProver, Box<dyn StdError>> {
let container_name = port.map(|p| format!("sp1-gpu-{p}")).unwrap_or("sp1-gpu".t
let image_name = std::env::var("SP1_GPU_IMAGE")
.unwrap_or_else(|_| "public.ecr.aws/succinct-labs/moongate:v5.0.8".to_string());
let gpus = visible_device_index.map(|i| format!("device={i}")).unwrap_or("all".
// Docker 컨테이너 시작
Command::new("docker")
.args([
"run",
"-e", &format!("RUST_LOG={rust_log_level}"),
"-p", &format!("{port}:3000"),
"--rm",
"--gpus", &gpus,
"--name", &container_name,
```

```
&image_name,
])
.spawn()?;
}
```

증명 생성 프로세스

CUDA 가속화된 증명 생성은 다음 단계로 진행됩니다:

1. Core Proof 생성

```
pub fn prove_core(&self, stdin: &SP1Stdin) -> Result<SP1CoreProof, SP1CoreProv
let payload = ProveCoreRequestPayload { stdin: stdin.clone() };
let request = crate::proto::api::ProveCoreRequest {
data: bincode::serialize(&payload).unwrap()
};
let response = block_on(async { self.client.prove_core(request).await }).unwrap
let proof: SP1CoreProof = bincode::deserialize(&response.result).unwrap();
Ok(proof)
}
```

2. 압축 (Compression)

```
pub fn compress(
```

```
&self,
vk: &SP1VerifyingKey,
proof: SP1CoreProof,
deferred_proofs: Vec<SP1ReduceProof<InnerSC>>,
) -> Result<SP1ReduceProof<InnerSC>, SP1RecursionProverError> {
let payload = CompressRequestPayload { vk: vk.clone(), proof, deferred_proofs }
let request = crate::proto::api::CompressRequest {
data: bincode::serialize(&payload).unwrap()
};
let response = block_on(async { self.client.compress(request).await }).unwrap()
let proof: SP1ReduceProof<InnerSC> = bincode::deserialize(&response.result).unw
Ok(proof)
}
```

3. 축소 (Shrink)

```
pub fn shrink(
&self,
reduced_proof: SP1ReduceProof<InnerSC>,
) -> Result<SP1ReduceProof<InnerSC>, SP1RecursionProverError> {
let payload = ShrinkRequestPayload { reduced_proof: reduced_proof.clone() };
let request = crate::proto::api::ShrinkRequest {
data: bincode::serialize(&payload).unwrap()
};
```

```
let response = block_on(async { self.client.shrink(request).await }).unwrap();
let proof: SP1ReduceProof<InnerSC> = bincode::deserialize(&response.result).unw
Ok(proof)
}
```

4. BN254 래핑

```
pub fn wrap_bn254(
&self,
reduced_proof: SP1ReduceProof<InnerSC>,
) -> Result<SP1ReduceProof<OuterSC>, SP1RecursionProverError> {
let payload = WrapRequestPayload { reduced_proof: reduced_proof.clone() };
let request = crate::proto::api::WrapRequest {
data: bincode::serialize(&payload).unwrap()
};
let response = block_on(async { self.client.wrap(request).await }).unwrap();
let proof: SP1ReduceProof<OuterSC> = bincode::deserialize(&response.result).unw
Ok(proof)
}
```

메모리 관리 방식

GPU 메모리 요구사항

SP1 CUDA 가속화는 상당한 GPU 메모리를 요구합니다:

- **최소 VRAM**: 24GB
- **권장 VRAM**: 32GB 이상
- **메모리 사용 패턴**: 증명 생성 단계별로 다른 메모리 요구량

메모리 최적화 전략

1. **단계별 메모리 관리**
   - 각 증명 생성 단계마다 필요한 메모리만 할당
   - 단계 완료 후 즉시 메모리 해제
2. **배치 처리**
   - 여러 증명을 배치로 처리하여 메모리 효율성 향상
   - GPU 메모리 대역폭 최적화
3. **데이터 압축**
   - 네트워크 전송을 위한 데이터 직렬화/압축
   - bincode를 사용한 효율적인 직렬화

성능 향상 효과

벤치마킹 시스템

SP1은 성능 측정을 위한 전용 벤치마킹 도구를 제공합니다:

```
#[derive(Default, Debug, Clone)]
struct PerfResult {
pub  cycles: u64,
```

```
pub execution_duration: Duration,
pub prove_core_duration: Duration,
pub verify_core_duration: Duration,
pub compress_duration: Duration,
pub verify_compressed_duration: Duration,
pub shrink_duration: Duration,
pub verify_shrink_duration: Duration,
pub wrap_duration: Duration,
pub verify_wrap_duration: Duration,
}
```

CPU vs GPU 성능 비교

벤치마킹 도구를 통한 성능 측정 결과:

Core Proof 생성

- **CPU 모드**: 단일 스레드 기반 순차 처리
- **CUDA 모드**: 병렬 GPU 연산으로 대폭 가속화
- **예상 성능 향상**: 10-50배 (프로그램 복잡도에 따라 차이)

압축 및 재귀적 증명

- **메모리 집약적 연산**: GPU의 높은 메모리 대역폭 활용
- **병렬 처리**: 다수의 CUDA 코어를 통한 동시 연산
- **예상 성능 향상**: 5-20배

실제 사용 사례별 성능

1. **간단한 프로그램** (피보나치 등)

- CPU: 수 분
- GPU: 수십 초

2. **복잡한 암호화 연산**

- CPU: 수십 분 ~ 수 시간
- GPU: 수 분 ~ 수십 분

3. **대규모 프로그램**

- CPU: 수 시간 ~ 수일
- GPU: 수십 분 ~ 수 시간

환경 설정 및 사용법

필수 요구사항

1. **하드웨어**

- NVIDIA GPU (24GB+ VRAM)
- CUDA 지원 GPU

2. **소프트웨어**

- Docker 설치 및 실행 권한
- NVIDIA Docker 런타임

환경 변수 설정

```
# 프로버 모드 설정
export SP1_PROVER=cuda
```

```
# 사용할 GPU 디바이스 지정 (선택사항)
export CUDA_VISIBLE_DEVICES=0

# 커스텀 GPU 이미지 사용 (선택사항)
export SP1_GPU_IMAGE=custom-moongate:latest
```

프로그래밍 인터페이스

```
use sp1_sdk::{ProverClient, SP1Stdin};

// CUDA 프로버 클라이언트 생성
let client = ProverClient::builder().cuda().build();

// 프로그램 설정
let (pk, vk) = client.setup(elf);

// 증명 생성 (자동으로 GPU 가속화 적용)
let proof = client.prove(&pk, &stdin).run().unwrap();

// 검증
client.verify(&proof, &vk).unwrap();
```

오류 처리 및 폴백 메커니즘

자동 폴백

CUDA 가속화가 실패할 경우 자동으로 CPU 모드로 폴백:

```
impl CudaProver {
pub fn prove_with_fallback(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin) -> Res
match self.cuda_prover.prove_core_stateless(pk, stdin) {
Ok(proof) => Ok(proof),
Err(cuda_error) => {
tracing::warn!("CUDA proving failed, falling back to CPU: {}", cuda_error);
self.cpu_prover.prove(pk, stdin, SP1ProofMode::Core)
}
}
}
}
```

일반적인 오류 상황

1. **Docker 관련 오류**

- Docker 미설치 또는 권한 부족
- GPU 드라이버 문제

2. **메모리 부족**

- GPU VRAM 부족

- 시스템 메모리 부족

3. **네트워크 오류**

- 컨테이너 통신 실패
- 포트 충돌

최적화 권장사항

하드웨어 최적화

1. **GPU 선택**

- RTX 4090, A100, H100 등 고성능 GPU 권장
- 최소 24GB VRAM 필요

2. **시스템 구성**

- 충분한 시스템 RAM (32GB+)
- 빠른 SSD 스토리지

소프트웨어 최적화

1. **배치 크기 조정**

- GPU 메모리에 맞는 적절한 배치 크기 설정
- 메모리 사용량과 성능의 균형점 찾기

2. **컨테이너 설정**

- 적절한 Docker 리소스 할당
- GPU 디바이스 매핑 최적화

3. **네트워크 최적화**

- 로컬 네트워크 사용으로 지연시간 최소화

- 적절한 포트 설정

결론

SP1의 CUDA 가속화는 Docker 컨테이너 기반의 Moongate 서버를 통해 구현되며, gRPC 통신을 통해 GPU 기반 증명 생성을 제공합니다. 이를 통해 CPU 대비 10-50배의 성능 향상을 달성할 수 있으며, 자동 폴백 메커니즘을 통해 안정성도 보장합니다. 적절한 하드웨어와 환경 설정을 통해 대규모 프로그램의 증명 생성 시간을 크게 단축할 수 있습니다.